



Universidad autónoma de baja California

Ingeniería en computación

Ingeniería en software

Propuesta de proyecto traductores

Guillermo Licea Sandoval

Garcia Chávez Erik 01275863

Martes 11 de noviembre del 2025

Interprete de pseudocódigo "PseudoThinkRun"

El objetivo principal de este proyecto es diseñar e implementar un programa capaz de analizar, validar y ejecutar algoritmos escritos en un lenguaje de pseudocódigo simple, estructurado y orientado a la educación.

El intérprete procesará archivos de texto plano que contengan código fuente, ejecutará las instrucciones secuencialmente y producirá una salida en la consola, simulando el comportamiento de un lenguaje de programación real.

El pseudocódigo es una herramienta fundamental en la enseñanza de la programación y la algoritmia. Permite a los estudiantes centrarse en la lógica y la estructura de una solución sin preocuparse por la sintaxis compleja de un lenguaje de programación formal. Un intérprete funcional de pseudocódigo refuerza este aprendizaje, proporcionando retroalimentación instantánea y permitiendo la depuración de la lógica del algoritmo.

Objetivo General:

Desarrollar un intérprete funcional para un subconjunto de pseudocódigo en español, que cumpla con todas las fases de un traductor (análisis léxico, sintáctico y semántico) y sea capaz de ejecutar programas que incluyan variables, estructuras de control y funciones.

Objetivos Específicos:

- Implementar un **analizador lexicográfico** (lexer) para tokenizar el código fuente.
- Implementar un **analizador sintáctico** (parser) para construir un Árbol de Sintaxis Abstracta (AST) que represente la estructura del programa.
- Implementar una **tabla de símbolos** para gestionar el alcance (scope) y la información de variables y funciones.
- Realizar un **análisis semántico** básico (ej. comprobación de tipos) durante el recorrido del AST.
- Desarrollar el **módulo intérprete** que recorra el AST validado y ejecute la lógica del programa.
- Cumplir con los requisitos específicos del proyecto: manejo de declaración de variables, los enunciados **"Repite"** y **"Mientras"**, y la declaración e invocación de funciones.

El lenguaje " PseudoThinkRun " será un lenguaje imperativo, estructurado y con tipado estático simple. A continuación, se definen sus características principales:

Tipos de Datos:

El intérprete soportará cuatro tipos de datos básicos:

- **Entero:** Para números enteros (ej. 10, -5).
- **Real:** Para números con punto decimal (ej. 3.1416).
- **Cadena:** Para texto (ej. "Hola").
- **Logico:** Para valores booleanos (Verdadero, Falso).

Declaración de Variables:

Toda variable deberá ser declarada explícitamente antes de su uso, utilizando la sintaxis requerida:

Definir mi_variable, otra_variable Como Entero

Estructuras de Control:

Se implementarán las siguientes estructuras de control:

- **Condicional:** Si <condicion> Entonces ... [Sino ...] FinSi
- **Bucle (Requerido):** Repite <expresion_numerica> veces ... FinRepite
- **Bucle (Adicional):** Mientras <condicion> Hacer ... FinMientras

Funciones:

El lenguaje soportará la declaración e invocación de funciones, un requisito clave del proyecto:

- **Declaración:** Funcion mi_funcion(param1, param2) ... Retornar valor
FinFuncion
- **Invocación:** resultado = mi_funcion(10, "hola")
- Las funciones tendrán su propio alcance (scope) para variables locales y parámetros.

Operaciones y Salida:

- Se soportarán operadores aritméticos (+, -, *, /), relacionales (==, !=, <, >) y lógicos (Y, O, NO).
- La salida por consola se manejará con el comando: Escribir expresion1, expresion2, ...

El intérprete se construirá siguiendo las fases canónicas de un traductor:

1. **Fase 1: Análisis Lexicográfico (Scanner):** El código fuente en texto plano se lee y se descompone en una secuencia de *tokens* (ej. PALABRA_RESERVADA_DEFINIR, IDENTIFICADOR, NUMERO_ENTERO, OPERADOR_SUMA).
2. **Fase 2: Análisis Sintáctico (Parser):** El parser recibe la secuencia de tokens y verifica que sigan la gramática definida para "PseudoThinkRun". El resultado de esta fase es un **Árbol de Sintaxis Abstracta (AST)**. Este AST es la representación intermedia del código (cumpliendo con el requisito de "generación de código intermedio").
3. **Fase 3: Análisis Semántico y Tabla de Símbolos:**
 - **Tabla de Símbolos:** Se crea y mantiene una estructura de datos para rastrear todas las variables y funciones declaradas, sus tipos y sus ámbitos (global o local a una función). Esta fase es **explícitamente requerida** por el proyecto.
 - **Análisis Semántico:** El intérprete recorre el AST para validar la semántica del programa. Esto incluye:
 - Verificar que las variables se declaren antes de usarse.
 - Comprobar la concordancia de tipos en asignaciones y operaciones (ej. no sumar un Entero con una Cadena directamente).
 - Validar el número y tipo de argumentos en las llamadas a funciones.
4. **Fase 4: Ejecución (Intérprete):** Una vez que el AST ha sido validado semánticamente, un "visitante" (visitor pattern) recorre el árbol por segunda vez. En esta ocasión, en lugar de validar, ejecuta las acciones. Por ejemplo, al visitar un nodo de asignación, actualiza el valor en la tabla de símbolos; al visitar un nodo Si, evalúa la condición y decide qué rama del árbol visitar a continuación.

Lenguaje de Implementación: Se propone el uso de **Java** (versión 11 o superior, preferentemente una LTS) debido a su robustez, su fuerte sistema de tipos y su ecosistema maduro para el desarrollo de traductores.

Este proyecto resultará en un intérprete de pseudocódigo robusto y funcional, capaz de ejecutar una variedad de algoritmos educativos. El desarrollo cubrirá de manera integral todos los requisitos obligatorios del proyecto final, proporcionando una base sólida en la teoría y práctica de los traductores de lenguajes.

